

Практическая работа: «AJAX-интерактивность каталога»

Темы: Клиентский JavaScript: DOM, события, Fetch API, Promise / async-await. Без jQuery — только vanilla JS.

Общее описание:

Вы берёте готовый каталог из прошлых практик (модели, layout, Bootstrap-оформление) и добавляете клиентскую динамику: поиск без перезагрузки, добавление в корзину через AJAX, обновление счётчика. Серверный код на C# и Razor уже есть — вы дописываете методы в контроллер и пишете JS-скрипты.

Задание 1. AJAX-эндпоинты на сервере

Добавьте в существующий `CatalogController` три метода. Они отдадут не целые страницы, а только данные — HTML-фрагмент (partial) или JSON.

1.1. Метод `Search` — отдаёт HTML-фрагмент

```
[HttpGet]
public IActionResult Search(string query)
{
    var products = _repo.GetAll();

    if (!string.IsNullOrEmpty(query))
    {
        query = query.ToLower();
        products = products.Where(p =>
            p.Name.ToLower().Contains(query) ||
            p.Description.ToLower().Contains(query));
    }

    return PartialView("_ProductList", products);
}
```

Что делает: фильтрует товары на сервере, возвращает `partial _ProductList.cshtml` — тот же HTML с карточками, но без layout. Нужно создать `Views/Shared/_ProductList.cshtml` (или переименовать существующий partial) — это просто `@foreach` по товарам с карточками Bootstrap.

1.2. Метод `AddToCart` — отдаёт JSON

```
[HttpPost]
public IActionResult AddToCart(int id)
{
    var product = _repo.GetById(id);
    if (product == null)
        return Json(new { success = false, message = "Товар не найден" });

    // Простая корзина в памяти через Session
    var cart = HttpContext.Session.GetString("Cart");
    var items = string.IsNullOrEmpty(cart)
        ? new List<int>()
        : cart.Split(',').Select(int.Parse).ToList();

    items.Add(id);
    HttpContext.Session.SetString("Cart", string.Join(',', items));

    return Json(new
    {
        success = true,
        cartCount = items.Count,
        productName = product.Name
    });
}
```

Что делает: кладёт ID товара в корзину (через сессию), возвращает JSON-объект с количеством товаров и названием добавленного. В Program.cs нужно добавить `builder.Services.AddSession()` и `app.UseSession()`.

1.3. Метод `GetCartCount` — отдаёт JSON

```
[HttpGet]
public IActionResult GetCartCount()
{
    var cart = HttpContext.Session.GetString("Cart");
    var count = string.IsNullOrEmpty(cart) ? 0 : cart.Split(',').Length;
    return Json(new { count = count });
}
```

Что делает: возвращает текущее количество товаров в корзине — для обновления бейджа при загрузке страницы.

Проверка: Открыть браузер, ввести `/Catalog/Search?query=ноут` в адресной строке и увидеть HTML-фрагмент без layout. Ввести `/Catalog/GetCartCount` — увидеть JSON `{"count":0}`.

Задание 2. Подготовка HTML-разметки

Прежде чем писать JS, нужно убедиться, что в Razor-страницах есть элементы с правильными `id` и `data-*-атрибутами`.

2.1. На странице каталога — `Views/Catalog/Index.cshtml`

Добавьте поле поиска и оберните сетку товаров в контейнер с `id`:

```
<input type="text" id="searchInput" class="form-control mb-4"
      placeholder="Поиск товаров..." />

<div id="catalogGrid" class="row">
    @foreach (var product in Model)
    {
        <partial name="_ProductCard" model="product" />
    }
</div>
```

2.2. В partial карточки товара — `Views/Shared/_ProductCard.cshtml`

Кнопка «В корзину» должна содержать `data-product-id`:

```
<div class="col-md-4 mb-4">
    <div class="card h-100">
        
        <div class="card-body">
            <h5 class="card-title">@Model.Name</h5>
            <p class="card-text">@Model.Price.ToString("C")</p>
            <button class="btn btn-primary w-100 add-to-cart"
                data-product-id="@Model.Id">
                В корзину
            </button>
        </div>
    </div>
</div>
```

2.3. В navbar — _Layout.cshtml

Бейдж корзины с id, чтобы JS мог его найти:

```
<a class="nav-link" href="/Cart">
  Корзина
  <span class="badge bg-danger" id="cartBadge">0</span>
</a>
```

Проверка: показать, где находятся id="searchInput", id="catalogGrid", id="cartBadge" и data-product-id — и объяснить, зачем они нужны для JS.

Задание 3. Live-поиск без перезагрузки

Файл: wwwroot/js/search.js

Пошагово:

Шаг 1. Найти элементы DOM:

```
const input = document.getElementById('searchInput');
const grid = document.getElementById('catalogGrid');
```

Шаг 2. Подписаться на событие ввода текста:

```
input.addEventListener('input', function () {
  // этот код выполняется при каждом нажатии клавиши
});
```

Шаг 3. Добавить дебаунс — задержку 300 мс, чтобы запрос уходил не на каждый символ, а только когда пользователь перестал печатать:

```
let timeoutId;

input.addEventListener('input', function () {
  clearTimeout(timeoutId); // отменить предыдущий таймер
  const query = input.value.trim();

  if (query.length < 2) {
    // если ввели 0-1 символ — перезагружаем страницу, чтобы показать все товары
    window.location.reload();
    return;
  }

  // запускаем новый таймер — выполнится через 300 мс
  timeoutId = setTimeout(function () {
    searchProducts(query);
  }, 300);
});
```

Шаг 4. Написать функцию searchProducts с состояниями загрузка / ошибка / результат:

```
async function searchProducts(query) {
  // 1. Показать спиннер
  grid.innerHTML =
    '<div class="text-center py-5">' +
    '<div class="spinner-border text-primary"></div>' +
    '</div>';

  try {
    // 2. Отправить запрос к серверу
    const url = '/Catalog/Search?query=' + encodeURIComponent(query);
    const response = await fetch(url);
```

```

    if (!response.ok) {
        throw new Error('HTTP ' + response.status);
    }

    // 3. Получить HTML из ответа
    const html = await response.text();

    // 4. Проверить, пустой ли результат
    if (!html.trim()) {
        grid.innerHTML =
            '<div class="alert alert-info">Ничего не найдено</div>';
    } else {
        // 5. Вставить карточки в сетку
        grid.innerHTML = html;
    }
} catch (error) {
    // 6. Показать сообщение об ошибке
    grid.innerHTML =
        '<div class="alert alert-danger">Ошибка поиска</div>';
    console.error(error);
}
}
}

```

Проверка: ввести текст, увидеть спиннер, потом результаты. Объяснить зачем encodeURIComponent, почему async/await, что будет без try/catch.

Задание 4. Добавление в корзину через AJAX

Файл: wwwroot/js/cart.js

Шаг 1. Найти все кнопки «В корзину» и подписаться на клик:

```

const buttons = document.querySelectorAll('.add-to-cart');

buttons.forEach(function (btn) {
    btn.addEventListener('click', function () {
        // data-product-id="5" → dataset.productId = "5"
        const productId = this.dataset.productId;
        addToCart(productId, this);
    });
});

```

Шаг 2. Функция addToCart:

```

async function addToCart(productId, button) {
    // 1. Сохранить исходный текст кнопки
    const originalText = button.textContent;

    // 2. Заблокировать кнопку и показать спиннер
    button.disabled = true;
    button.innerHTML =
        '<span class="spinner-border spinner-border-sm"></span> Добавление...';

    try {
        // 3. Отправить POST-запрос
        const response = await fetch('/Catalog/AddToCart', {
            method: 'POST',
            headers: {
                'Content-Type': 'application/x-www-form-urlencoded'
            },
            body: 'id=' + productId
        });
    }
}

```

```

    if (!response.ok) {
        throw new Error('HTTP ' + response.status);
    }

    // 4. Разобрать JSON-ответ
    const data = await response.json();

    if (data.success) {
        // 5. Обновить бейдж корзины
        const badge = document.getElementById('cartBadge');
        badge.textContent = data.cartCount;

        // 6. Показать успех на кнопке
        button.innerHTML = 'Добавлено ✓';
        button.classList.remove('btn-primary');
        button.classList.add('btn-success');

        // 7. Через 2 секунды вернуть исходное состояние
        setTimeout(function () {
            button.textContent = originalText;
            button.classList.remove('btn-success');
            button.classList.add('btn-primary');
            button.disabled = false;
        }, 2000);
    } else {
        alert('Ошибка: ' + data.message);
        button.textContent = originalText;
        button.disabled = false;
    }
} catch (error) {
    console.error(error);
    button.textContent = 'Ошибка';
    button.disabled = false;

    setTimeout(function () {
        button.textContent = originalText;
    }, 2000);
}
}

```

Проверка: кликнуть «В корзину», увидеть спиннер на кнопке, бейдж обновляется, кнопка становится зелёной на 2 секунды. Объяснить: почему POST, зачем disabled, что в data.cartCount.

Задание 5. Обновление счётчика корзины при загрузке

Файл: wwwroot/js/site.js

При загрузке любой страницы нужно запросить у сервера текущее количество товаров в корзине и обновить бейдж.

```

document.addEventListener('DOMContentLoaded', async function () {
    try {
        const response = await fetch('/Catalog/GetCartCount');
        const data = await response.json();

        const badge = document.getElementById('cartBadge');
        if (data.count > 0) {
            badge.textContent = data.count;
        } else {
            badge.textContent = '0';
        }
    } catch (error) {
        console.error('Не удалось получить количество товаров:', error);
    }
});

```

Проверка: добавить товары, перезагрузить страницу — бейдж сохраняет значение. Объяснить, зачем запрос при загрузке.

Задание 6. Подключение скриптов

6.1. Глобальный скрипт — в `_Layout.cshtml`

`site.js` подключается на всех страницах (нужно для бейджа):

```
<environment include="Development">
  <script src="~/lib/bootstrap/dist/js/bootstrap.bundle.js"></script>
  <script src="~/js/site.js" asp-append-version="true"></script>
</environment>
<environment exclude="Development">
  <script src="~/lib/bootstrap/dist/js/bootstrap.bundle.min.js"></script>
  <script src="~/js/site.min.js" asp-append-version="true"></script>
</environment>
```

6.2. Скрипты каталога — в `Index.cshtml` через секцию

`search.js` и `cart.js` нужны только на странице каталога:

```
@section Scripts {
  <script src="~/js/search.js" asp-append-version="true"></script>
  <script src="~/js/cart.js" asp-append-version="true"></script>
}
```

6.3. В `bundleconfig.json` — добавить новые файлы

```
[
  {
    "outputFileName": "wwwroot/js/site.min.js",
    "inputFiles": [ "wwwroot/js/site.js" ]
  },
  {
    "outputFileName": "wwwroot/css/site.min.css",
    "inputFiles": [ "wwwroot/css/site.css" ]
  }
]
```

Проверка: объяснить, почему `search.js` и `cart.js` подключаются через секцию, а `site.js` — глобально.

Структура файлов

wwwroot/

```
├── js/
│   ├── site.js      ← обновление бейджа при загрузке (глобально)
│   ├── search.js    ← live-поиск (только на странице каталога)
│   └── cart.js       ← добавление в корзину (только на странице каталога)
```

Views/

```
├── Shared/
│   ├── _Layout.cshtml ← navbar с id="cartBadge", site.js
│   ├── _ProductCard.cshtml ← кнопка с data-product-id
│   └── _ProductList.cshtml ← partial для AJAX-ответа (поиск)
└── Catalog/
    └── Index.cshtml      ← id="searchInput", id="catalogGrid", @section Scripts
```

Controllers/

```
└── CatalogController.cs ← Search, AddToCart, GetCartCount
```

Дополнительные задания (на оценку «отлично»)

Уровень: Лёгкий

Toast-уведомление при добавлении в корзину

Вместо смены текста кнопки покажите всплывающее уведомление Bootstrap toast.

Что нужно сделать:

1. В `_Layout.cshtml` добавьте HTML-структуру toast (скрытую по умолчанию):

```
<div class="position-fixed top-0 end-0 p-3" style="z-index: 11">
  <div id="cartToast" class="toast align-items-center text-white bg-success
    border-0" role="alert">
    <div class="d-flex">
      <div class="toast-body" id="cartToastText">
        Товар добавлен
      </div>
      <button type="button" class="btn-close btn-close-white me-2 m-auto"
        data-bs-dismiss="toast"></button>
    </div>
  </div>
</div>
```

2. В `cart.js` после успешного ответа — показать toast:

```
const toastEl = document.getElementById('cartToast');
document.getElementById('cartToastText').textContent =
  'Товар «' + data.productName + '» добавлен в корзину';

const toast = new bootstrap.Toast(toastEl, { delay: 3000 });
toast.show();
```

Что демонстрирует: работа с Bootstrap JS-компонентами из своего JS-кода, `setTimeout` (через `delay`), DOM-манипуляции.

Уровень: Средний

Бесконечная подгрузка (Infinite Scroll)

При прокрутке каталога до конца — автоматически подгружается следующая порция товаров.

Что нужно сделать:

1. Сервер: метод `Index` принимает параметр `page` (по 6 товаров на страницу). Новый метод `LoadMore(int page)` возвращает `partial` со следующей порцией.
2. В `Index.cshtml` добавьте «сентинель» — пустой `div` в конце списка:

```
<div id="sentinel" style="height: 20px;"></div>
```

3. В `search.js` (или новом `infinite.js`) — наблюдатель:

```
let currentPage = 1;
let isLoading = false;
let hasMore = true;

const observer = new IntersectionObserver(async function (entries) {
  if (entries[0].isIntersecting && !isLoading && hasMore) {
    isLoading = true;
    currentPage++;

    const url = '/Catalog/LoadMore?page=' + currentPage;
    const response = await fetch(url);
    const html = await response.text();

    if (html.trim()) {
      grid.insertAdjacentHTML('beforeend', html);
    } else {
      hasMore = false;
      observer.disconnect();
    }

    isLoading = false;
  }
});

observer.observe(document.getElementById('sentinel'));
```

Что демонстрирует: `IntersectionObserver`, `insertAdjacentHTML`, флаг состояния `isLoading`.

Уровень: Сложный

Модальное окно с динамической загрузкой через `Promise.all`

Клик по карточке открывает модальку, содержимое которой загружается из двух параллельных AJAX-запросов.

Что нужно сделать:

1. Сервер: два метода:
 - a) `GetProductDetails(int id)` — JSON: `{ name, price, description, imageUrl }`
 - b) `GetProductReviews(int id)` — JSON: `[{ author, text, rating }, ...]`
2. В `Index.cshtml` — пустая модалька (структура `Bootstrap modal` с пустым телом и спиннером внутри).
3. В `cart.js` (или новом `product-modal.js`):

```
async function openProductModal(productId) {
  // 1. Открыть модальку со спиннером
  const modalBody = document.getElementById('modalBody');
  modalBody.innerHTML = '<div class="spinner-border"></div>';
```

```

bootstrap.Modal.getOrCreateInstance(
    document.getElementById('productModal')).show();

try {
    // 2. Два запроса параллельно
    const [detailsResp, reviewsResp] = await Promise.all([
        fetch('/Catalog/GetProductDetails?id=' + productId),
        fetch('/Catalog/GetProductReviews?id=' + productId)
    ]);

    // 3. Распарсить детали (обязательно)
    const product = await detailsResp.json();

    // 4. Распарсить отзывы (может не получиться — это OK)
    let reviews = [];
    try {
        const reviewsData = await reviewsResp.json();
        reviews = reviewsData;
    } catch {
        // отзывы не загрузились — товар всё равно показываем
    }

    // 5. Собрать HTML вручную
    let html = '<h3>' + product.name + '</h3>';
    html += '<p class="fw-bold">' + product.price + '</p>';
    html += '<p>' + product.description + '</p>';
    html += '<hr><h5>Отзывы</h5>';

    if (reviews.length > 0) {
        html += '<ul class="list-group">';
        reviews.forEach(function (r) {
            html += '<li class="list-group-item">' +
                '<strong>' + r.author + '</strong> (' + r.rating + '/5)<br>' +
                r.text + '</li>';
        });
        html += '</ul>';
    } else {
        html += '<p class="text-muted">Отзывов пока нет</p>';
    }

    modalBody.innerHTML = html;
} catch (error) {
    modalBody.innerHTML =
        '<div class="alert alert-danger">Не удалось загрузить данные</div>';
}
}

```

Что демонстрирует: Promise.all, постройка HTML из JSON в JS, устойчивость к частичным ошибкам (товар показывается даже без отзывов), управление модалькой через JS.